МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных технологий

Кафедра
управления и информатики
в технических системах

ОТЧЕТ О ВЫПОЛНЕНИИ
ЛАБОРАТОРНОЙ РАБОТЫ ПО ДИСЦИПЛИНЕ

«Прикладное программирование»
по теме «Регулярные выражения. Unit-тестирование»

СТУДЕНТА 2 КУРСА бакалавриата ГРУППЫ ИДБ-25-12
(уровень профессионального образования)

Баязитова Ильсина Юлдыбаевна

Направление: 09.03.03 Прикладная информатика

Отчет сдан «_____» _____2026г.

Оценка _____

Преподаватель Гетманов С. В. _____
(Ф.И.О., должность, степень, звание.) *(подпись)*

МОСКВА 2026

Оглавление

1. Цель работы 3

2. Постановка задачи 3

3. Описание предметной области 3

4. Разработка регулярного выражения 4

5. Реализация программы 7

5.1. Консольный интерфейс 8

6. Unit-тестирование 9

7. Работа с системой контроля версий Git 11

8. Заключение 11

1. Цель работы

Цель лабораторной работы – изучить регулярные выражения в Python и модуль re, научиться составлять собственное регулярное выражение для проверки и поиска данных в пользовательском вводе, текстовом файле и на веб-странице, а также освоить unit-тестирование с помощью модуля unittest.

2. Постановка задачи

Необходимо выполнить следующие задачи:

– составить собственное регулярное выражение (вариант не из предложенного списка) и реализовать его на Python;

– предусмотреть проверку и поиск данных в строке, введённой пользователем, в загруженном файле и на веб-странице, полученной по URL (библиотека requests);

– написать unit-тесты для проверки корректности работы регулярного выражения и остального кода;

– вести разработку в системе контроля версий Git и оформить отчёт.

3. Описание предметной области

Выбран собственный вариант регулярного выражения: «Проверка и поиск географических координат в десятичных градусах». Тема продолжает предметную область первой лабораторной работы «Прокат велосипедов»: у каждой станции проката есть положение на карте, а координаты станций в реальных данных (текстовые файлы, страницы сайтов, сообщения) записываются в разных форматах и с ошибками.

Программа должна распознавать два формата записи координаты (сначала широта, затем долгота) и отбрасывать записи, выходящие за допустимые пределы. Правила формализованы в таблице 1.

Таблица 1. Правила записи координаты

Элемент	**Правило**
Формат 1	широта и долгота в десятичных градусах со знаком «+» или «−» (знак необязателен), разделитель – запятая или точка с запятой, вокруг разделителя допустимы пробелы: 55.7558, 37.6176
Формат 2	число, знак градуса «°», буква полушария; допустимы N/S/E/W и русские С/Ю/В/З; между широтой и долготой запятая, точка с запятой или пробел: 55.7558° N, 37.6176° E
Широта	от 0 до 90 по модулю; для значения 90 дробная часть только из нулей
Долгота	от 0 до 180 по модулю; для значения 180 дробная часть только из нулей
Дробная часть	обязательна, от 1 до 8 цифр; разделитель – точка
Ведущие нули	не допускаются (05.7558 – неверно)
Полушария	S и Ю (юг), W и З (запад) дают отрицательное значение; N, С, В, E – положительное
Границы при поиске	координата не должна «прилипать» к другим символам: слева не допускаются буква, цифра, точка, «+», «−»; справа – буква, цифра и точка с цифрой

Примеры корректных записей: «55.7558, 37.6176», «-33.8688; 151.2093», «33.8688° S, 151.2093° E», «33.8688° Ю, 70.6693° З». Примеры некорректных записей: «91.5, 37.6» (широта больше 90), «55.7, 181.2» (долгота больше 180), «55, 37» (нет дробной части), «55,7558, 37,6176» (запятая вместо точки), «55.7558° E, 37.6176° N» (перепутаны полушария).

4. Разработка регулярного выражения

Регулярное выражение собрано из частей с помощью f-строк, поэтому каждая часть имеет имя и описана отдельно. Составные части приведены в таблице 2.

Таблица 2. Составные части регулярного выражения

Часть	**Шаблон**	**Что описывает**
LAT_NUMBER	(?:90\.0{1,8}|[1-8]?\d\.\d{1,8})	число от 0.0 до 90.0 с дробной частью
LON_NUMBER	(?:180\.0{1,8}|1[0-7]\d\.\d{1,8}|[1-9]?\d\.\d{1,8})	число от 0.0 до 180.0 с дробной частью
Формат 1	(?P<lat>[+-]?LAT)\s*[,;]\s*(?P<lon>[+-]?LON)	координата со знаком
Формат 2	(?P<lat_h>LAT)\s*°\s*(?P<ns>[NSСЮ])\s*[,;]?\s*(?P<lon_h>LON)\s*°\s*(?P<ew>[EWВЗ])	координата с полушарием
Левая граница	(?<![\w.+-])	отрицательный просмотр назад
Правая граница	(?!\w)(?!\.\d)	отрицательный просмотр вперёд

Для широты число 0–89 описывает ветка [1-8]?\d (одна цифра или две, первая от 1 до 8), а значение 90 вынесено в отдельную ветку 90\.0{1,8}, чтобы запретить 90.5. Аналогично для долготы: ветка 1[0-7]\d покрывает 100–179, ветка [1-9]?\d – 0–99, а 180 вынесено отдельно. Благодаря этому проверка диапазона выполняется самим регулярным выражением, без дополнительного сравнения чисел.

Именованные группы (?P<lat>…) позволяют извлечь значения без подсчёта номеров скобок. Так как Python не допускает одинаковых имён групп, во втором формате использованы имена lat_h, lon_h, ns, ew. Выражение компилируется один раз функцией re.compile и применяется методами

fullmatch (проверка всей строки) и finditer (поиск в тексте). Общая структура выражения показана на рисунке 1, итоговый фрагмент кода – ниже.

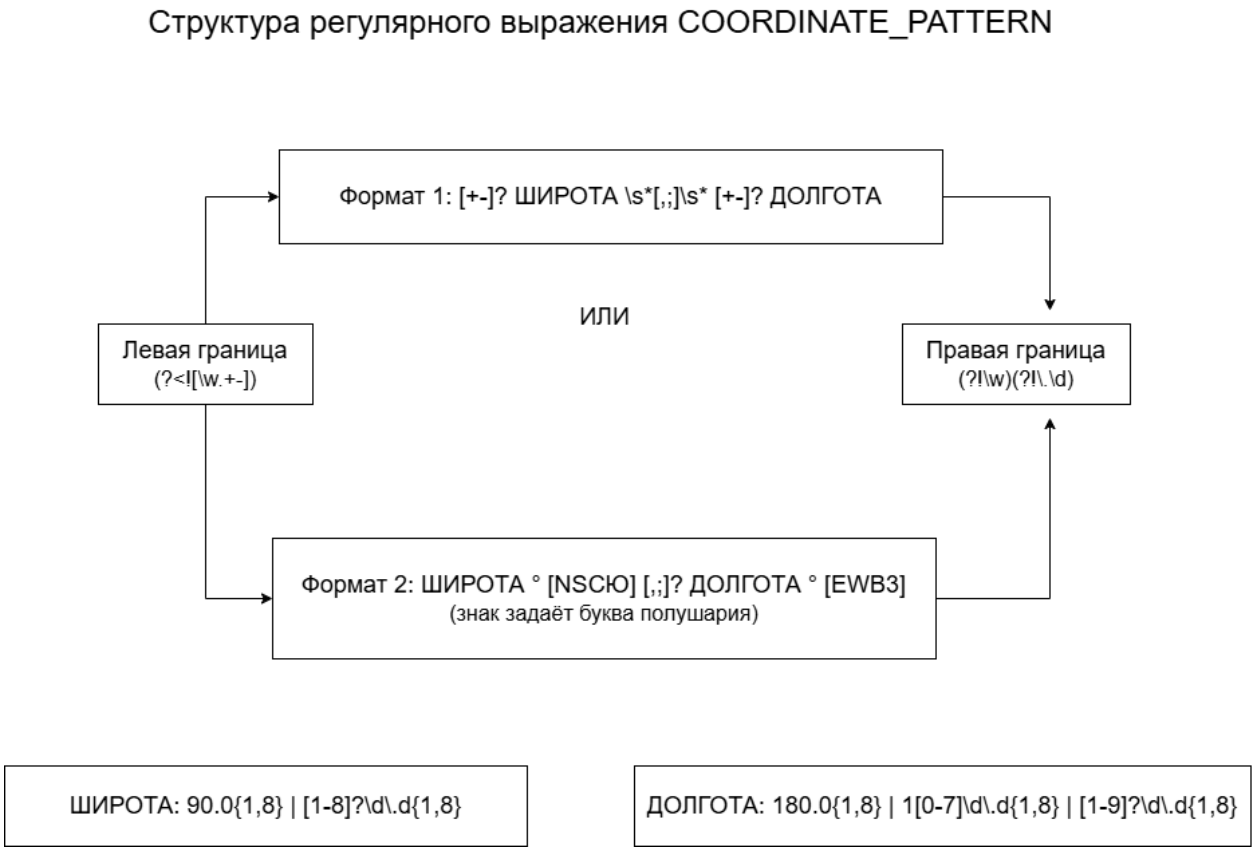


Рис. 1. Структура регулярного выражения

```
# Широта: от 0 до 90 включительно, обязательная дробная часть (1-8 цифр).
LAT_NUMBER = r"(?:90\.0{1,8}|[1-8]?\d\.\d{1,8})"
# Долгота: от 0 до 180 включительно, обязательная дробная часть (1-8 цифр).
LON_NUMBER = r"(?:180\.0{1,8}|1[0-7]\d\.\d{1,8}|[1-9]?\d\.\d{1,8})"


# Формат 1: знак перед числом, разделитель - запятая или точка с запятой.
_SIGNED = (
    rf"(?P<lat>[+-]?{LAT_NUMBER})\s*[,;]\s*(?P<lon>[+-]?{LON_NUMBER})"
)
# Формат 2: знак градуса и буква полушария после каждого числа.
_HEMISPHERE = (
    rf"(?P<lat_h>{LAT_NUMBER})\s*°\s*(?P<ns>[NSСЮ])"
    rf"\(?:\s*[,;]\s*|\s+"
    rf"(?P<lon_h>{LON_NUMBER})\s*°\s*(?P<ew>[EWВЗ])"
)


# Слева не должно быть буквы, цифры, точки, плюса или минуса,
# справа - буквы/цифры или точки с цифрой (чтобы не резать длинные числа).
COORDINATE_PATTERN = re.compile(
    rf"(?<![\w.+-])(?:{_SIGNED}|{_HEMISPHERE})(?!\w)(?!\.\d)"
)
```

5. Реализация программы

Программа написана на Python и разделена на три модуля по ответственности (таблица 3). Регулярное выражение вынесено в отдельный модуль и не зависит от источника данных, поэтому один и тот же код работает для пользовательского ввода, файла и веб-страницы.

Поток данных между модулями показан на рисунке 2.



Рис. 2. Поток данных программы

Таблица 3. Структура проекта

Файл	**Назначение**
coordinates.py	регулярное выражение COORDINATE_PATTERN, класс Coordinate (широта, долгота, исходная запись), функции is_valid_coordinate, parse_coordinate, find_coordinates, исключение InvalidCoordinateError

Файл	Назначение
sources.py	read_text_file – чтение файла; fetch_url_text – загрузка страницы через requests; html_to_text – удаление скриптов, стилей и тегов методом re.sub и раскрытие HTML-сущностей; исключение SourceError
app.py	консольное меню; ошибки источников и ввода перехватываются, программа продолжает работу
data/stations.txt	примеры входных данных (в том числе с намеренно ошибочными записями)
tests/	test_coordinates.py, test_sources.py, test_app.py

Функция read_text_file открывает файл в текстовом режиме с кодировкой UTF-8 и преобразует FileNotFoundError, UnicodeDecodeError и OSError в собственное исключение SourceError с понятным сообщением. Функция fetch_url_text проверяет схему URL (только http и https), выполняет requests.get с тайм-аутом, проверяет код состояния методом raise_for_status и также преобразует ошибки HTTP, соединения и тайм-аута в SourceError. Перед поиском из страницы удаляются теги, а также содержимое тегов script и style, чтобы числа из кода и стилей не принимались за координаты.

5.1. Консольный интерфейс

Взаимодействие с пользователем организовано через консольное меню, графический интерфейс не используется. Ниже приведён реальный сеанс работы: проверка корректной и некорректной строки, поиск в введённом тексте, поиск в файле data/stations.txt, ошибки при отсутствующем файле и неверном URL. После каждой ошибки программа возвращается в меню.

```
Выберите пункт: 1
Строка: 55.7558, 37.6176
Корректно: 55.755800, 37.617600
Выберите пункт: 1
Строка: 91.5, 37.6
Некорректная координата
Выберите пункт: 2
Текст: Станция 55.7522, 37.5927 и 33.8688° S, 151.2093° E
Найдено координат: 2
1. 55.7522, 37.5927 -> 55.752200, 37.592700
2. 33.8688° S, 151.2093° E -> -33.868800, 151.209300
Выберите пункт: 3
Путь к файлу: data/stations.txt
Найдено координат: 5
1. 55.755800, 37.617600 -> 55.755800, 37.617600
2. 55.7312° N, 37.6014° E -> 55.731200, 37.601400
```

```
3. 55.7101; 37.5537 -> 55.710100, 37.553700
4. -55.9833, -67.2667 -> -55.983300, -67.266700
5. 33.8688° S, 151.2093° E -> -33.868800, 151.209300
Выберите пункт: 3
Путь к файлу: none.txt
Ошибка: Файл none.txt не найден
Выберите пункт: 4
URL: http://
Ошибка: Некорректный URL: http://
Выберите пункт: abc
Нет такого пункта меню
Выберите пункт: 0
До свидания!
```

Поиск по веб-странице выполняется после выбора пункта 4 и ввода URL. Программа загружает страницу с помощью библиотеки requests, удаляет HTML-теги, содержимое элементов script и style, а также преобразует HTML-сущности в соответствующие символы. После этого выполняется поиск координат в полученном тексте. Таким образом, программа позволяет находить координаты на внешних веб-страницах без использования локального HTML-файла.

6. Unit-тестирование

Тесты написаны с помощью стандартного модуля unittest, всего 56 тестов (таблица 4). Для проверки таблиц данных использован контекст subTest, поэтому при ошибке видно конкретный проблемный пример. Сетевые запросы в тестах заменены заглушками (unittest.mock.patch), поэтому тесты не зависят от интернета, а для файловых тестов используются временные каталоги.

Таблица 4. Состав тестов

Тестовый класс	**Тестов**	**Что проверяется**
TestIsValidCoordinate	7	корректные записи обоих форматов; граничные значения 90/180; выход широты и долготы за диапазон; неверный формат чисел (нет дробной части, ведущие нули, 9 знаков после точки, запятая вместо точки); неверная структура (пустая строка, лишние символы, перепутанные полушария, строчные буквы); нестроковые значения

Тестовый класс	Тестов	Что проверяется
TestParseCoordinate	8	значения и знаки при разборе, полушария S/Ю/W/З, исходная запись, строковое представление, равенство объектов, исключение InvalidCoordinateError
TestFindCoordinates	14	пустой текст, текст без координат, координата в предложении, в скобках и кавычках, смешанные форматы и порядок, повторы, многострочный текст, пропуск некорректных записей и «прилипших» чисел (IP-адреса, версии)
TestReadTextFile	5	чтение файла, отсутствующий файл, неверная кодировка, каталог вместо файла, другая кодировка
TestHtmlToText	4	удаление тегов, скриптов и стилей, регистр и переносы строк, HTML-сущности
TestFetchUrlText	5	успешная загрузка, ошибка HTTP 404, ошибка соединения, тайм-аут, некорректные URL без обращения к сети
TestMenu	13	выход, неизвестный пункт, все четыре пункта меню, ошибки файла и URL, непредвиденная ошибка, EOFError и KeyboardInterrupt

Результат запуска тестов командой python -m unittest discover -v (сокращённый вывод):

```
test_valid_signed (tests.test_coordinates.TestIsValidCoordinate) ... ok
test_valid_hemisphere (tests.test_coordinates.TestIsValidCoordinate) ... ok
test_latitude_out_of_range (tests.test_coordinates.TestIsValidCoordinate) ...
ok
test_longitude_out_of_range (tests.test_coordinates.TestIsValidCoordinate) ...
ok
test_bad_number_format (tests.test_coordinates.TestIsValidCoordinate) ... ok
test_bad_structure (tests.test_coordinates.TestIsValidCoordinate) ... ok
test_find_by_url (tests.test_app.TestMenu) ... ok
test_missing_file_does_not_stop_program (tests.test_app.TestMenu) ... ok
test_success (tests.test_sources.TestFetchUrlText) ... ok
test_connection_error (tests.test_sources.TestFetchUrlText) ... ok
...
----------------------------------------------------------------------
Ran 56 tests in 0.008s

OK
```

Покрытие кода измерено по строкам: coordinates.py – 100 %, sources.py – 100 %, app.py – 97 % (не выполняется только строка запуска run() под условием __name__ == "__main__"). Для повторной проверки можно использовать команды coverage run -m unittest discover и coverage report -m.

Код оформлен с соблюдением PEP 8 (длина строк не более 79 символов, docstring у модулей и функций, осмысленные имена), содержит комментарии к неочевидным местам.

7. Работа с системой контроля версий Git

Лабораторная работа размещена в папке lab2 общего репозитория учебных работ. Разработка велась по этапам, каждый этап фиксировался отдельным коммитом: модуль coordinates.py с регулярным выражением; модуль sources.py; консольное меню app.py; тестовые данные; unit-тесты; README и отчёт.

Ссылка на репозиторий GitHub: https://github.com/nexus-100/labs (папка lab2).

8. Заключение

В ходе лабораторной работы разработано собственное регулярное выражение для проверки и поиска географических координат в двух форматах записи (десятичные градусы со знаком и с обозначением полушария). Допустимые диапазоны широты (0–90) и долготы (0–180) контролируются самим выражением. Программа на Python позволяет проверять и искать координаты в пользовательском вводе, текстовом файле и на веб-странице, загружаемой через библиотеку requests.

Написано 56 unit-тестов, проверяющих основные функции программы. Общее покрытие кода по строкам составляет 99%: coordinates.py – 100%, sources.py – 100%, app.py – 97%. Поставленные задачи лабораторной работы выполнены.